

不要相信人性，只要相信体系/制度

制造业品质自治 Playbook

作者：颜士尊/Terence Gan · 从业二十余年的品质与运营管理者

目录

00 序：一个品质管理者的诚实坦白 01 问题：制造业品质管理的「改了又改」死循环 02 第一性原理：不要相信人性，只要相信体系/制度 03 多视角交叉验证 04 机器锚验真 05 宁可误拦，不放过 06 防假绿：表层齐全、里子空 07 边做边公布 08 从眼馋到体系的路线图 09 诚实边界：带伤疤的生产产品

00 序：一个品质管理者的诚实坦白

我做了二十多年品质与运营，管过多家工厂。我自认不是「技术大神」，而是「被问题逼出来的管理者」。

写这个 Playbook 之前，我先坦白三件事：

第一，这里没有「完美成品」。我下面讲的每一套方法，都是「真跑过生产、带着伤疤」的——它们不是从书里抄的，是踩了坑、赔了钱、返了工之后逼出来的。伤疤是深度的证据，也是诚实的声明：它们有用，但不完美。

第二，我一度以为「招到好人、靠人自觉」就能管好品质。后来发现这是最大的幻觉——好人也会累、会忘、会自以为是、会被自己的「第一印象」带着走。我把这句话当成我这套方法的地基：不要相信人性，只要相信体系/制度。

第三，这套方法里最反直觉的一点，我写在方法五：把你不成熟的东西「公开」出去，反而能逼你把事情做对——因为会被看到，所以不敢糊弄。

01 问题：制造业品质管理的「改了又改」死循环

品质管理的老大难，不是「没数据」，是「有数据、有制度，但就是改不好」。

最典型的现象是「改了又改」：同一个问题，修了又犯、犯了又修，循环往复。每次修完都觉得「这次肯定好了」，过一阵子又原地复发。

为什么？根子不在「没努力」，在三个地方：

一是「靠自觉」。制度写在墙上，但执行靠人记、靠人自觉。人一忙、一换、一累，制度就形同虚设。

二是「靠单一视角」。一个问题，一个人看觉得「修好了」，换个人看就发现「还漏了一块」。单点判断，天然有盲区。

三是「看表层」。很多「闭环」是纸面的——报告写了、签字有了、绿灯亮了，但底下什么都没变。

这三个根子，合起来就是一句话：我们把「品质」寄托在了「人性」上，而人性是最不可靠的零件。

02 第一性原理：不要相信人性，只要相信体系/制度

这句话是我这套方法的全部地基。

「人性」是什么？是自觉、是记忆、是诚实、是「我以为我做到了」。而这一切，都会衰减、会漂移、会自欺——而且最要命的是，自欺的时候人自己是意识不到的。

「体系/制度」是什么？是流程、是机器可校验的客观依据、是「不做就不算完成」的强制动作、是「会被看到所以不敢糊弄」的公开。

举一个我亲身验证过的例子（脱敏）：我曾经让一个人把四个人的意见「汇总」成一份建议。他以为自己忠实转写了，结果漏了其中三个人各说的一条——因为他在汇总时，被其中一个人的思路带着走，把其他三个人的不同意见「顺手过滤」了。他自己完全没意识到。后来我加了一条「制度」：汇总时必须列一张对账表，每个人的每条意见要么收录、要么写明「不收录的原因」——从那以后，这种「无意识过滤」就藏不住了。

这就是「体系/制度 > 人性」的全部意思：不是人不可信，而是人（包括 AI、包括我）的「自觉」不可信；把「自觉」换成「机器可校验的制度」，事情才对得上。

03 多视角交叉验证

问题：为什么一个人看不全

人看东西有个毛病——第一个印象会像船锚一样，把后面的判断死死钉住，这叫"锚定"。一旦锚住，人就只找支持第一印象的证据，对相反的证据视而不见。一个人看一张表、一份报告，漏掉的东西他自己是看不见的，因为"看不见"这件事本身他也看不见。

方法：让几个独立视角各自看，再对账

不要一个人从头看到尾。把同一件事拆成几个独立视角，每个视角只从自己的角度独立看，互不商量、互不影响。看完了再放到一起对——对得上的地方放心，对不上的地方就是问题所在。关键在"独立"：如果几个视角先开了会、统一了口径，那就等于又变回了一个人看。

一个抽象案例

四个人各从不同角度审查同一份材料，得出四条结论，其中三条指向同一个风险，一条说没问题。负责汇总的人把四条合成一份时，先看到了那条"没问题"的意见，被它的框架锚住，于是默默把另外三条"有风险"的结论当成噪音过滤掉了。最后交出去的报告看起来工整，实际上漏掉了四个人里三个人的判断。这不是谁故意使坏，就是锚定——先看到什么，就被什么牵着走。多视角的意义，就是逼着自己看见本来会漏掉的东西。

04 机器锚验真

问题："自报"为什么不可靠

人和机器都有一个毛病——会"自报"。人说自己看过了、转写忠实了；机器（AI）也说自己检查过了、没问题。但"自报"靠不住：人会过度自信，会无意识地过滤掉自己不想看见的东西；AI 更麻烦，它会"自报假绿"——嘴上说查了，其实没真查。你不能问当事人"你做对了吗"，因为他一定说对。

方法：用"机器锚"验真，不靠自报

"机器锚"就是一份可校验的客观依据，它不依赖任何人的主观陈述。比如一条铁规则："每一条结论，都必须在原始清单里能找到归处。"再比如一张"对账表"：左边是原始材料里的条目，右边是转写后的结论，两边一条一条对。对上了才算数，对不上就是漏了。判断对错靠对账，不靠"我觉得我转写对了"。

一个抽象案例

一个汇总者要转写四个人的意见，转写完后信誓旦旦说"我忠实转写了"。别人没有直接信他，而是拿出原始清单一条一条对：四个人的意见一共七条，转写出来的只有四条，漏了三条。要不是有这张对账表，漏掉的三条就永远消失了——因为转写者自己坚信"我都写了"。机器锚的作用就在这里：它不质问你的信心，它只对账。

05 宁可误拦，不放过

问题：两种错，代价不一样

一个检查环节会犯两种错：一是"放水"——该拦的没拦，坏东西流了下去；二是"误拦"——该放的拦了，好东西被多审了一轮。这两种错都不好，但代价天差地别。"放水"是让错误无声无息地流到下游，等你发现的时候，它已经混进成品里、传到了客户手里，挽回成本极高；"误拦"只是多费一道人工复核，成本有限、可控。

方法：fail-closed，默认拦住

所以制度设计要"宁可误拦、不放过"：检查器默认不放行，证据充分才放。宁可把一百件好东西拦下来重新看一遍，也不放过去一件坏东西。放行必须有明确的、可校验的证据支撑，而不是"看起来应该没问题"。宁可多审一轮，也不要赌一把。

一个抽象案例

一条流水线末尾有一道检查门。以前这道门"看情况放行"，偶尔有坏件混过去，到了客户那里才被发现，返工、赔钱、还丢了信任。后来改了规矩：这道门默认关闭，只有拿到了"合格证据"才开门。结果平均每件东西多花几分钟重新核一遍，但坏件从此到不了客户手里。多花的那几分钟，和一次漏放造成的损失比，完全不是一个量级。这就是"宁可误拦"的账。

06 防假绿

问题：看着做了，其实没做

最危险的不是没做，是"看着做了、其实没做"——表层齐全、里子空。检查器每天照常运行、照常报"通过"，报告一份不少、格式漂漂亮亮，闭环也走完了签字盖章。但掀开看，检查器从来没真正拦下过任何东西，报告是复制粘贴的，闭环是纸面上的。这种"假闭环"比不做更危险，因为它给了所有人"我们已经管住了"的错觉，让人放下警惕。

方法：用"注入已知错误"来测试

怎么识破假绿？主动试探。往系统里塞一个明知是错的样本，看它会不会被拦住。如果检查器对错误样本照样报"通过"，说明它是摆设，从来没真正工作过。真检查器一定会在错误样本上变红。这个方法叫"注入已知错误验证"——不给它讲道理，直接用一道错题考它。

一个抽象案例

一个检查环节运行了很久，每天都报"一切正常"。有人起了疑心，故意往要检查的数据里塞了一个明显的错误（一个不可能出现的数值）。结果检查环节照样报"通过"，绿灯亮起，错误大摇大摆地过了。这一下就露了底：它不是真在检查，只是在走流程。于是从头查，才发现这个检查环节的判定逻辑早就失效了，一直在空转。没有这一道错题，这个空转的摆设可能还会"正常"运转很多年。

07 边做边公布

很多人做事喜欢闭门憋大招，想着等做得足够好、足够完整，再拿出来给人看。这个想法听起来稳妥，其实是个坑。闭门做的最大问题，是没人看见，糊弄的成本极低。今天偷个懒、明天省个步骤，反正没人知道，日子照样过。

反过来，一边做一边公布，情况就完全不一样。只要你说过要做、别人也在看，你就不好意思糊弄。因为会被看到，所以不敢偷懒；因为会被追问，所以不敢把没验证的东西说成已经做成了。「被看到」带来的压力，比任何自觉都管用。

所以「公开」本身就是制度的一种。这本书讲「不要相信人性，只要相信体系/制度」，很多人以为制度就是流程、表格、审批。其实「让事情被看见」也是一种制度，而且是成本最低、见效最快的一种。

你把自己晾在众人面前做，自然就认真了。别人看着你做，你自然不敢停下。这就是边做边公布的价值：公开不是结果，是逼你变好的手段。

08 从眼馋到体系的路线图

看到别人做得好，第一反应往往是眼馋，想直接抄过来。眼馋不可耻，但眼馋到建立自己的体系，中间还有几步路，一步都不能省。

第一步，识破眼馋。要分清两件事：你是看上了别人那个具体的东西，还是看到别人在追你早就想追的方向、只是提醒了你「这事该对外了」。前者是抄作业，后者才是方向。

第二步，提炼成方法论。把别人做得好的地方，抽掉具体数字、具体客户、具体工具，剩下能复用的道理。能讲给一个外行人听懂，才算提炼成功；讲出来全是术语，说明还没消化。

第三步，固化成制度。方法论靠自觉是留不住的，今天记着明天就忘。要把它变成一个必须走的流程：到了时间就得输出，输出完就得检查。不靠人记性，靠流程逼着做。

第四步，对外公开。把前面几步的成果拿出去给人看，用「被看到」倒逼自己持续做下去，别让它烂在抽屉里。

眼馋只解决「想不想要」，体系才解决「怎么一直有」。两者之间差的，就是这四步。

09 诚实边界：带伤疤的生产品

全章定位：全书最后一道刹车。把「哪些已验证、哪些还在演进」讲清楚，把「照搬会翻车」的坑指出来，把数据安全红线亮出来，再做一次发布前的全篇安全审计。

开篇：这不是「完美成品」，是「带伤疤的生产品」

一句话把话说清楚：这本书里讲的东西，不是实验室里打磨出来的完美展示品，而是真的在工厂里跑过、摔过跟头、留过疤的生产品。

「带伤疤」有两层意思，都得诚实承认。第一层，伤疤是真跑过生产的证据——一个从来没上过生产线的方案，是不会有疤的，因为它压根没遇到过真问题。第二层，伤疤也是「还没到完美」的声明——它还在长，有些地方还在改，我们不会把它吹成「一次到位、永不翻车」。

所以请先把预期摆正：这本书不是一本「照着做就一定成」的武功秘籍，而是一本「别人怎么摔、怎么爬起来、哪些坑别踩」的过来人手记。伤疤不是羞耻，是我们敢把没擦干净的地方亮给你看。

9.1 哪些已验证，哪些还在演进

诚实分两级，一条条说。

已经在生产里跑、每天都在用的（已验证）： - 自动化的数据闭环：数据进来、自动清洗、自动出报告，中间不用人抄数。 - 多视角交叉验证：同一个数，让不同角度、不同来源各自算一遍，对不上就拦下来。 - 机器锚验真：用客观标准当「锚」，不靠人嘴上说「我检查过了」。 - 宁可误拦、不可漏放的门禁：查不过就停，不放行。 - 防假绿审计：专门查「表面齐全、里子是空」的情况。

这些是「已验证」，可以放心讲。

还在演进、不敢打包票的（未验证）： - 专属领域模型还在持续训练迭代，不报分数、不报准确率——因为还在变。 - 多厂覆盖的自动化程度，厂和厂不一样，有的环节还是人工在顶。

诚实讲清楚「哪些已经稳、哪些还不稳」，比拍胸脯更值钱。

9.2 「照搬会翻车」的警告：诊断学 ≠ 处方学

这一节是全书的「刹车」，请务必读完。

这套方法论拆开是两样东西，性质完全不同：

第一样叫「诊断学」——帮你看出问题在哪的清单。比如「这个数有没有经过两个独立来源核对过」「这条线是机器在拦，还是靠人自觉」。这类清单有普适价值，拿去就能用，因为它只是逼你睁眼看清现状，不替你下药。

第二样叫「处方学」——照方抓药的解决方案。这类要格外小心，照搬最容易翻车。

最典型的翻车，就是以为「加更多 AI、加更多思考、加更多验证」是解药。错。如果这些「更多」全都来自同一个源头、同一个视角、同一批人，加得越多，回声室就越大声，错得越自信——这不是解药，是把偏见放大。

真正的解药只有三条：1. **锚定客观真值**——先立一个外部客观标准，人和机器都往它靠，而不是互相点赞。2. **跨模型、跨视角独立**——让不同的脑子各自独立算，再对答案，绝不能让它们抄同一份作业。3. **人工抽验兜底**——机器跑得再顺，也得有人定期随机抽几单，亲眼看一眼。

三条里任何一条被省掉，前面所有「方法」都可能变成自嗨。

9.3 数据安全红线：这条线，一个字都不让步

写这本书之前，我们先给自己划了一条红线，现在把这条线原样交给你：

所有真实品质数据——报废率、客诉、扣款金额、客户料号——一律不出境、不上公网。这本书只讲方法论，不碰任何一个真实数字、一个真实客户、一个真实料号。

为什么这么较真？因为方法论可以分享，数据是命根子。报废率是高是低、客诉几起、扣了多少钱、给哪家客户供货——这些数字一旦流出去，轻则丢单，重则违约。所以：

- 这本书里出现的所有数字、所有「案例」，都是虚构的、示意用的，没有一个能对上真实的工厂、真实的客户。
- 书里写「某工厂」「某客户」这种说法，不是藏，是我们压根不会写真名。

数据安全底线，是整套体系里唯一一条「宁可保守、绝不松动」的线。方法论可以给你，数据不会。

9.4 全篇安全审计（发布前最后一道关）

对外版交付前，按三条标准把全篇从头到尾审了一遍，结论如下：

第一条：零真实品质数据、零客户料号。 审计结论：通过。全篇没有任何报废率数值、客诉数量、扣款金额、客户名称或料号。文中出现的「报废率」「客诉」等词，仅作为「禁止泄露的类别」被点名，不是数据本身。

第二条：不泄露内部治理细节。 审计结论：通过（发布前须完成一处清理）。对外版正文未出现内部系统的具体名称、内部工具名、内部模型代号、内部人员代号，也未出现内部审核流程的步骤细节。需提醒：本书的**工作底稿**（草稿的目录、分工表）里还留着内部代号，发布前必须把这些代号换成通用表述（如「品质管理中心」）后再出街。对外版正文本身是干净的。

第三条：不把「带伤疤」吹成「完美」。 审计结论：通过。全篇未发现把「还在演进」说成「已经做成」的过度承诺，凡未验证的能力均以「持续迭代中」「自动化程度逐厂不同」等诚实口径呈现，没有拍胸脯、打包票的字眼。

一句话总结：这本书是诚实的——它敢把伤疤亮出来，也敢把「还没做到」的地方说清楚。你可以不信它的每一项结论，但你可以信它没有骗你。